

Universidad de Costa Rica
CI-0117 Programación Paralela y Concurrente
Prof. Josef Ruczicka González
Tarea 0
Jimena Bejarano Sánchez C31074
Marina Castro Peralta C31886

1. Red de Petri

La sincronización del sistema se modeló mediante una red de Petri en dos vistas. La primera muestra el ciclo de vida de un counter (sólo Economy, como ejemplo); las otras dos clases siguen el mismo patrón. La segunda vista muestra el sistema completo con las tres clases, las transiciones del Balancer y los ciclos de break/reopen del Supervisor.

Diagrama 1 — Ciclo de vida de un counter

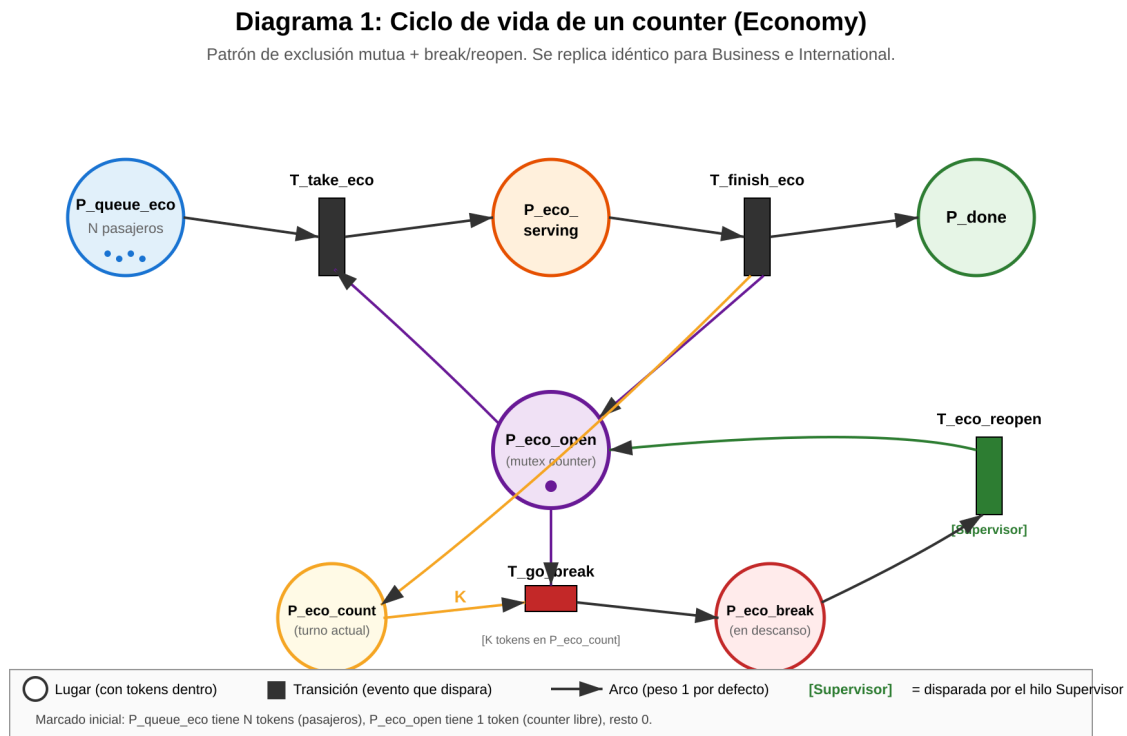


Figura 1. Ciclo de vida de un counter Economy: patrón de exclusión mutua + break/reopen.

Diagrama 2 — Vista global (3 clases + Balancer + Supervisor)

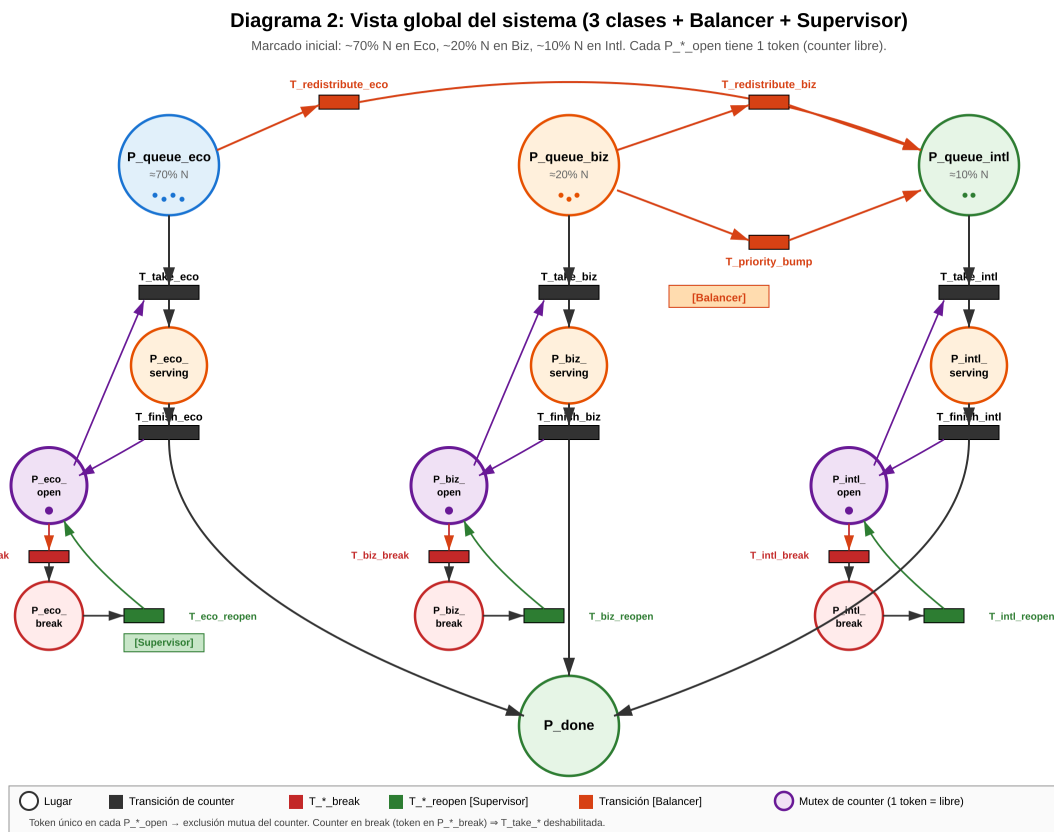


Figura 2. Vista global del sistema. Las transiciones del Balancer (en naranja) interconectan las tres colas; los ciclos de break/reopen están a los costados.

Tabla de lugares

ID	Nombre	Significado	Marcado inicial
P1	P_queue_eco	Pasajeros Economy esperando	≈ 70% de N
P2	P_queue_biz	Pasajeros Business esperando	≈ 20% de N
P3	P_queue_intl	Pasajeros International esperando	≈ 10% de N
P4	P_eco_serving	Pasajero siendo atendido por counter Economy	0
P5	P_biz_serving	Pasajero siendo atendido por counter Business	0
P6	P_intl_serving	Pasajero siendo atendido por counter International	0
P7	P_eco_open	Counter Economy disponible (mutex del counter)	1
P8	P_biz_open	Counter Business disponible (mutex del counter)	1
P9	P_intl_open	Counter International disponible (mutex del counter)	1

P1 0	P_eco_count	Pasajeros atendidos por counter Eco en turno actual	0
P1 1	P_biz_count	Pasajeros atendidos por counter Biz en turno actual	0
P1 2	P_intl_count	Pasajeros atendidos por counter Intl en turno actual	0
P1 3	P_eco_break	Counter Economy en break (no acepta pasajeros)	0
P1 4	P_biz_break	Counter Business en break (no acepta pasajeros)	0
P1 5	P_intl_break	Counter International en break (no acepta pasajeros)	0
P1 6	P_done	Pasajeros que ya completaron su check-in	0

Nota sobre el marcado inicial: el token único en cada P_*_open es la pieza clave del modelo. Representa al counter como recurso de exclusión mutua: mientras ese token no esté en su lugar, ninguna transición que lo requiera puede dispararse, lo que garantiza que no se puedan atender dos pasajeros simultáneamente en el mismo counter.

Tabla de transiciones

ID	Nombre	Disparada por	Precondición / guarda	Efecto
T 1	T_take_eco	Counter Economy	Tokens en P_queue_eco y P_eco_open	Pasa el pasajero a P_eco_serving
T 2	T_finish_eco	Counter Economy	Token en P_eco_serving	Pasajero a P_done, +1 en P_eco_count, devuelve token a P_eco_open
T 3	T_take_biz	Counter Business	Tokens en P_queue_biz y P_biz_open	Análogo a T1 para Business
T 4	T_finish_biz	Counter Business	Token en P_biz_serving	Análogo a T2 para Business
T 5	T_take_intl	Counter International	Tokens en P_queue_intl y P_intl_open	Análogo a T1 para International
T 6	T_finish_intl	Counter International	Token en P_intl_serving	Análogo a T2 para International
T 7	T_eco_break	Counter Economy	K tokens en P_eco_count y token en P_eco_open	Counter pasa a P_eco_break

T 8	T_biz_break	Counter Business	K tokens en P_biz_count y token en P_biz_open	Counter pasa a P_biz_break
T 9	T_intl_break	Counter International	K tokens en P_intl_count y token en P_intl_open	Counter pasa a P_intl_break
T 10	T_eco_reopen	Hilo Supervisor	Token en P_eco_break y tiempoFinBreak ya expiró	Devuelve token a P_eco_open
T 11	T_biz_reopen	Hilo Supervisor	Token en P_biz_break y tiempoFinBreak ya expiró	Devuelve token a P_biz_open
T 12	T_intl_reopen	Hilo Supervisor	Token en P_intl_break y tiempoFinBreak ya expiró	Devuelve token a P_intl_open
T 13	T_priority_bump	Hilo Balancer	Token en P_queue_biz con espera > T_max y bumpsEnVentana < 3	Mueve el token al frente de P_queue_intl
T 14	T_redistribute_eco	Hilo Balancer	P_queue_eco > Q	Mueve excedente al final de P_queue_intl
T 15	T_redistribute_biz	Hilo Balancer	P_queue_biz > Q	Mueve excedente al final de P_queue_intl

El valor de K para cada break se elige aleatoriamente en el rango [K_min, K_max] al inicio de cada turno del counter. La duración del break se sortea entre 500 ms y 2000 ms. La cuota anti-starvation bumpsEnVentana < 3 se reinicia cada 1000 ms.

2. Tabla de Lock Ordering

Para evitar deadlocks, el sistema establece un orden estricto de adquisición de mutex. Todo hilo que necesite tomar más de un lock lo hace siempre en este orden ascendente, y nunca al revés.

Mutex	Protege	Usado por	Nivel
cola.mutex (Eco / Biz / Intl)	Tamaño y nodos de cada cola FIFO; condition variable cond para señalar nuevos pasajeros	hiloCounter (al desencolar), hiloBalancer (al redistribuir o hacer bump)	1
counter[i].mutex	Estado del counter (OPEN/SERVING/ON_BREAK), estadísticas (totalAtendidos, sumaEsperaMs, sumaServicioMs), condition variable condReopen	hiloCounter (estado y stats), hiloSupervisor (al reabrir), main (lectura de totalAtendidos)	2

mutexActiva (global)	Flag activa que controla la vida de la simulación	main (al apagar la simulación), hiloCounter, hiloSupervisor, hiloBalancer (al leer)	Independiente
-------------------------	---	---	---------------

Reglas de uso del orden

- Las tres colas son del mismo nivel (1) y nunca se sostienen dos a la vez. Las operaciones del Balancer (redistribución y priority bump) trabajan en dos fases: extraen del origen bajo el lock de origen, lo sueltan, y luego encolan en el destino bajo el lock de destino. Nunca dos colas a la vez.
- El counter mutex (nivel 2) se toma siempre después de la cola, nunca al revés. El hiloCounter primero toma cola.mutex para desencolar al pasajero, lo suelta inmediatamente, y sólo después toma counter.mutex para actualizar estadísticas. Cola y counter nunca están simultáneamente locked.
- mutexActiva es independiente del resto. Se sostiene únicamente durante la lectura o escritura del flag activa, sin solicitar ningún otro lock mientras se sostiene. Por eso no participa en ningún orden global con los demás mutexes y es imposible que esté involucrado en un ciclo de espera.

Punto particular del hiloCounter en break: cuando un counter entra en break, mantiene counter.mutex y necesita esperar a que el Supervisor lo reabra. La espera se realiza con `pthread_cond_wait(&condReopen, &counter.mutex)`, que suelta automáticamente el mutex mientras el hilo está dormido y lo retoma al despertar. Esto evita cualquier riesgo de deadlock con el Supervisor (que también necesita el counter.mutex para hacer un signal) sin necesidad de juegos manuales de lock/unlock.

3. Argumento de correctitud

¿Por qué nuestro diseño no puede sufrir deadlock?

Una condición necesaria para que ocurra un deadlock es la espera circular en el grafo de dependencia de recursos. Demostramos que nuestro diseño no la admite al analizar los puntos en los que un hilo sostiene un lock y solicita otro.

Razón 1 — Orden global de mutexes. Todo el código siempre respeta el mismo orden de adquisición: cola.mutex (nivel 1) → counter.mutex (nivel 2). Ningún hilo adquiere un mutex de nivel superior mientras mantiene uno de nivel inferior. mutexActiva queda fuera de esta jerarquía porque nunca se mantiene mientras se solicita otro lock. Sin un ciclo de espera, un deadlock es imposible.

Razón 2 — Nunca se mantienen dos mutexes de la misma clase a la vez. El hiloCounter primero toma cola.mutex para sacar un pasajero, lo suelta completamente, y sólo después toma counter.mutex para actualizar estadísticas. El hiloBalancer en sus operaciones de redistribución y priority bump toma una sola cola a la vez (extrae del origen, suelta, encola en destino). El hiloSupervisor toma counter.mutex de un solo counter por vez. Esto descarta cualquier posibilidad de cadena cerrada de espera.

Verificación empírica. Esta propiedad fue verificada con Valgrind Helgrind en el servidor Kabré, que reporta cero errores de deadlock y cero lock-order inversions tras nuestras correcciones. En las primeras versiones del código Helgrind detectó 142 errores; tras los fixes la cuenta bajó a 0.

Por qué Economy no muere de hambre por los priority bumps de Business

Este fue uno de los grandes problemas iniciales con los que nos topamos. La starvation de Economy podría darse a través de dos mecanismos relacionados con el balancer: priority bumps masivos que llenen el frente de P_queue_intl con pasajeros Business, empujando indefinidamente a los Economy hacia atrás; o redistribuciones agresivas que saturen P_queue_intl y dejen sin atender a los pasajeros que llegaron primero.

El diseño previene ambos casos con tres mecanismos complementarios:

1. Cuota global de priority bumps. El Balancer no realiza más de 3 priority bumps por cada ventana de 1000 ms (parametrizado como MAX_BUMPS_POR_VENTANA y VENTANA_MS en balancer.c). Cuando se alcanza el cap, el Balancer omite cualquier bump adicional hasta que arranque la próxima ventana. Esto asegura que el frente de P_queue_intl se vacía periódicamente.
2. Un solo bump por iteración del Balancer. Aun dentro del cap, cada iteración del loop del Balancer hace a lo sumo un bump (la función revisarPriorityBump retorna inmediatamente después del primer movimiento). Esto distribuye los bumps en el tiempo en lugar de generar ráfagas.
3. La redistribución preserva FIFO de los que ya esperaban. La función redistribuirCola mueve el excedente del final de la cola origen, no del frente. Los pasajeros Economy que llegaron primero permanecen en su cola Economy original y siguen siendo atendidos por sus counters dedicados.

En conjunto, estos tres mecanismos garantizan que la cola P_queue_intl no quede dominada por pasajeros bumpeados, y que ningún pasajero Economy quede indefinidamente postergado. En las corridas de prueba con N=100 todos los pasajeros Economy fueron atendidos en tiempos finitos y razonables.

Por qué todos los pasajeros son atendidos exactamente una vez

El main duerme periódicamente con nanosleep(50 ms) y al despertar revisa si sumarAtendidos == N. La suma se hace tomando counter.mutex de cada counter para leer totalAtendidos de forma segura. Esto *no es busy-waiting*: el hilo está dormido entre revisiones (50 ms es mucho mayor al tiempo de un quantum de scheduling), no consume CPU activamente. Sólo cuando todos los N pasajeros han sido contabilizados, main toma mutexActiva, escribe activa = 0, lo libera, y luego despierta a todos los hilos bloqueados con pthread_cond_broadcast en cada cola y cada condReopen. Los hilos detectan que la simulación terminó, salen limpiamente, y main hace pthread_join de todos antes de liberar memoria.

4. Reflexión

Desafíos enfrentados

1. Diseño inicial de la arquitectura

El primer reto fue plantear cómo hacer que varios hilos trabajaran simultáneamente con tres tipos distintos de pasajeros (Economy, Business e International) que llegaban de forma mixta. La solución fue clasificarlos al crearlos y almacenarlos en un único arreglo continuo de Counter, lo que permitió manejar todos los counters con un solo loop, independientemente de su tipo. Esto simplificó tanto la creación de hilos como la recolección de estadísticas al final.

2. Data race en la variable activa

Durante el desarrollo, la variable activa se declaró como volatile int asumiendo que eso era suficiente para que los hilos la leyeran correctamente. Sin embargo, al correr Helgrind en Kabré se detectó un error de data race: el hilo main escribía activa = 0 al mismo tiempo que hiloCounter, hiloSupervisor e hiloBalancer la leían, todo sin ninguna protección. El problema no era visible en ejecución normal porque volatile sólo le dice al compilador que no optimice el

acceso, pero no garantiza exclusión mutua entre hilos. La solución fue introducir un mutex global `mutexActiva` que protege toda lectura y escritura de esa variable.

3. Entender en la práctica qué es sección crítica y dónde poner los mutexes

Aunque la lógica de "abrir y cerrar" un mutex parece simple, fue necesario analizar cuidadosamente qué datos se compartían entre hilos y en qué momentos. Por ejemplo, las estadísticas de cada counter (`totalAtendidos`, `sumaEsperaMs`) son escritas por el hilo del counter y leídas por el main, por lo que deben protegerse. También fue necesario entender que el mutex se puede tomar y soltar varias veces dentro de una misma función, dependiendo de qué datos se accede en cada momento, para minimizar el tiempo que se sostiene el lock y así reducir la contención.

4. Wakeups espurios y `cond_wait` con `while`

Al principio, el counter usaba un `if` antes del `pthread_cond_wait` para esperar el reopen del Supervisor. Después de leer la documentación de POSIX y volver a revisar el código, descubrimos que `cond_wait` puede regresar sin haber sido señalado (wakeups espurios), lo que podía hacer que el counter se abriera aún en break. La corrección fue cambiar el `if` por un `while` que vuelva a verificar la condición tras cada wakeup, que es el patrón estándar correcto.

5. Compatibilidad con el GCC viejo de Kabré

El servidor Kabré utiliza una versión antigua de GCC que por defecto compila en modo C89 y oculta extensiones POSIX como `clock_gettime`, `nanosleep` y `rand_r`. Tras varios intentos con flags como `-std=c99 -D_POSIX_C_SOURCE=200809L` (que también escondía algunos símbolos), llegamos a la combinación `-std=gnu99` que habilita C99 + extensiones GNU/POSIX por defecto. Esto cambió el Makefile para que compile uniformemente en cualquier máquina.

Posibles mejoras

Si se tuviera más tiempo, varias mejoras concretas serían interesantes de implementar:

- Unidades adaptativas en las estadísticas finales. Actualmente, todos los tiempos se muestran en milisegundos, lo cual es útil para valores pequeños, pero resulta difícil de leer cuando la simulación corre con `N` grande y los tiempos de espera superan los 10.000 ms. Lo ideal sería mostrar automáticamente, en segundos o minutos, cuando el valor lo amerite.
- Llegadas dinámicas con distribución de Poisson. En lugar de que todos los pasajeros existan al inicio, llegarían a las colas a lo largo del tiempo según un proceso de Poisson, lo que se ajustaría mejor al comportamiento real de un aeropuerto.
- Dashboard en tiempo real. Un hilo adicional que cada segundo imprima el tamaño de cada cola y el estado de cada counter (`OPEN`, `SERVING`, `ON_BREAK`), permitiendo ver la dinámica del sistema mientras corre en lugar de sólo mirar el log final.
- Simulación de fallas de counter. Un counter podría fallar y requerir un hilo de Mantenimiento que lo repare antes de que el Supervisor pueda reabrirlo. Esto agregaría una capa más de coordinación entre hilos y haría más realista el modelo.